



NED UNIVERSITY
OF ENGINEERING & TECHNOLOGY

OEL REPORT

Single-Cycle RISC-V Processor Design and Simulation in Logisim

Computer Organization and Design

Department of Computer and Information Systems Engineering

NED University of Engineering & Technology

Lab Session 14 / Complex Engineering Activity

```
add x5, x1, x2
sub x6, x5, x3
lw x7, 0(x6)
sw x7, 4(x5)
beq x1, x0, label
jal x0, label
label:
...
```



INSTRUCTION FORMAT

31	20	19	15	14	12	11	7	6	0
funct7	rs2	rs1	funct3	rd	opcode				
7	5	5	3	5	7				

Group Members

#	Name	Roll Number
1	Muhammad Muzammil Alam	CS-24308
2	Muhammad Ayaan Shaukat	CS-24307
3	Muhammad Owais-ul-Azam	CS-24309
4	ADEENA REHAN	CS-24016
5	IFRAH ZULFIQAR	CS-24021



Submitted To:
Mam Hina Danish



Date:
5/18/2026

REPORT BRIEF

This report presents the design and implementation of a single-cycle RISC-V processor built and tested in Logisim. The processor integrates the major datapath blocks required for instruction fetch, register access, immediate generation, arithmetic and logic execution, memory access, write-back, and branch control.

Design focus

The architecture was developed as a modular datapath. Each sub-circuit performs one clear role, while the main control and ALU control units coordinate data movement for R-type arithmetic, LW, SW, and BEQ instructions.

Item	Specification
Processor model	Single-cycle RISC-V datapath
Design tool	Logisim / Logisim Evolution style circuit simulation
Word size	32-bit data path and 32-bit instruction width
Register file	32 general-purpose registers, X0 to X31
Supported instructions	R-type arithmetic/logical operations, LW, SW, BEQ
Memory organization	16M x 32-bit instruction memory and 16M x 32-bit data memory

CONTENTS

01	Objective and Scope	Purpose of the processor design and supported instruction classes.
02	Top-Level Datapath	Overall single-cycle integration and signal flow.
03	Register File Module	Dual-read, single-write register storage subsystem.
04	Instruction Fetch Unit	Program Counter and instruction memory addressing.
05	Immediate Generator	I-type, S-type, and B-type immediate reconstruction.
06	Data Memory Module	Load and store memory interface.
07	Main and ALU Control Units	Opcode decoding and ALU operation selection.
08	Arithmetic Logic Unit	Execution unit, zero flag, and overflow flag.
09	Verification Summary	Final checks and expected behaviour.

1. OBJECTIVE AND SCOPE

The objective of this Open-Ended Lab is to implement a working single-cycle RISC-V processor and verify that the major datapath elements interact correctly. The design supports the core instruction categories needed for the lab: R-type arithmetic/logical instructions, load word, store word, and branch equal.

The report focuses on the structure of each module, the purpose of the main control signals, and how data moves through the processor during instruction execution. The implementation is divided into clean sub-circuits so that each unit can be tested independently and then connected inside the main datapath.

2. TOP-LEVEL SINGLE-CYCLE DATAPATH

The top-level datapath connects the Program Counter, instruction memory, register file, immediate generator, ALU, data memory, branch logic, and write-back multiplexer. In a single-cycle design, every instruction completes its fetch, decode, execute, memory, and write-back work within one clock cycle.

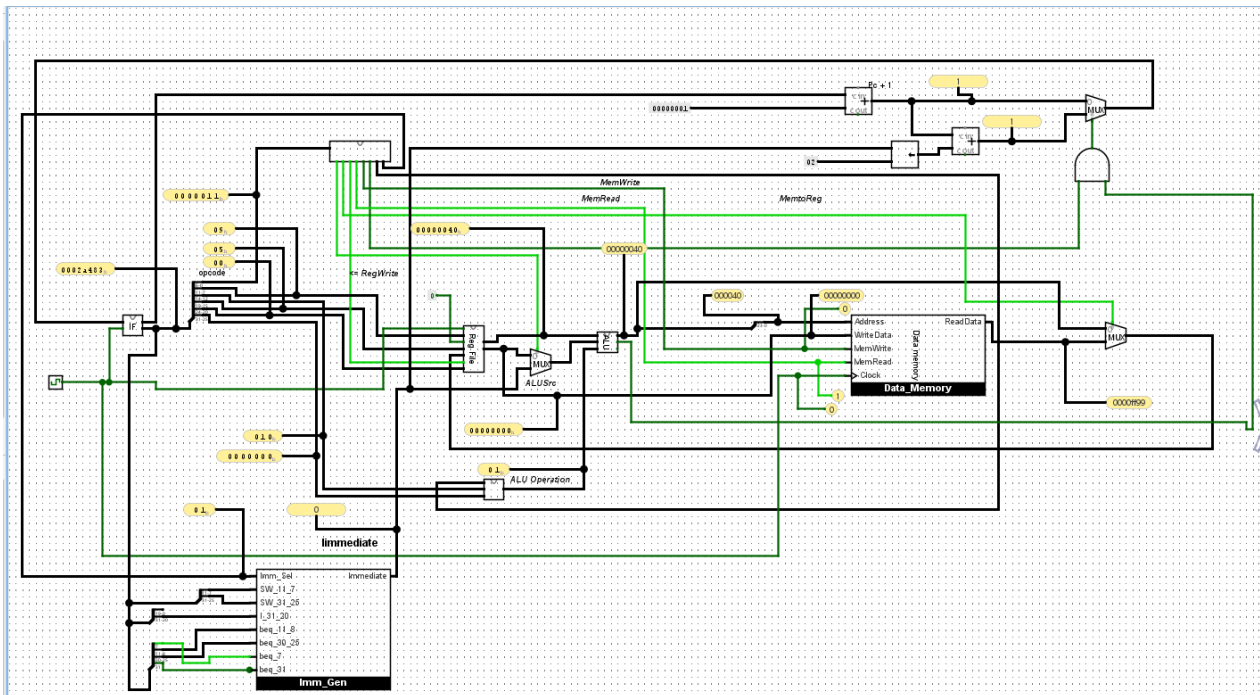


Figure 1: Complete single-cycle RISC-V datapath implemented in Logisim.

Datapath behaviour

For R-type instructions, the ALU result is written directly back to the destination register. For LW, the ALU calculates the address and memory data is written back. For SW, data from the second register is stored in memory. For BEQ, the zero flag and branch signal decide whether the next PC is PC + 4 or the branch target.

3. REGISTER FILE MODULE

The register file stores the architectural state of the processor. It contains 32 general-purpose 32-bit registers, named X0 through X31. The design provides two read ports for rs1 and rs2 and one write port for rd, allowing the processor to read two operands and write one result during an instruction cycle.

- **Storage elements:** The module is built from 32 individual 32-bit registers with reset and write-enable control.
- **Read network:** Multiplexers select the required register contents based on rs1_addr and rs2_addr.
- **Write network:** The destination register receives Write_Data only when RegWrite is active on the clock edge.

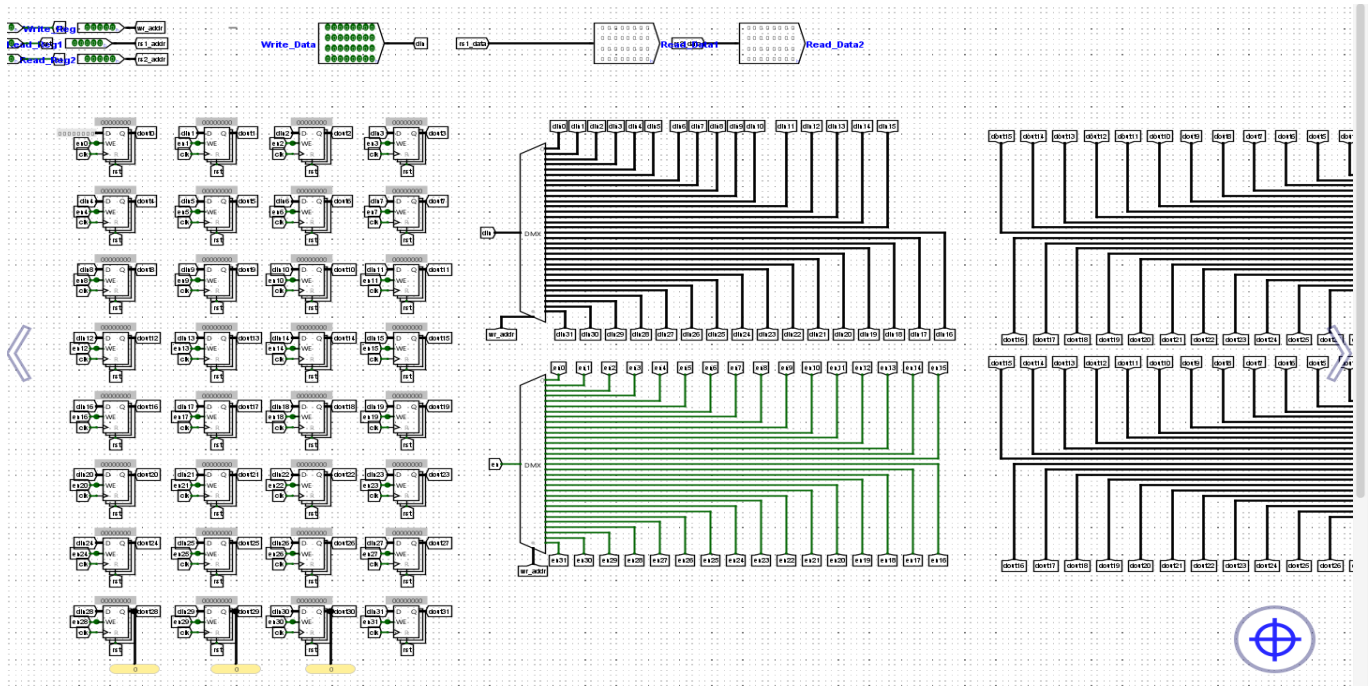


Figure 2: Register file sub-circuit with 32 registers and read multiplexers.

Item	Specification
Read ports	Two asynchronous read outputs: Read_Data1 and Read_Data2
Write port	One synchronous write input controlled by RegWrite
Register width	32 bits per register
Register count	32 registers, X0-X31

4. INSTRUCTION FETCH UNIT

The Instruction Fetch unit controls the current instruction address. The Program Counter holds the address of the instruction being executed and updates on the clock edge. The instruction memory then outputs the 32-bit machine instruction stored at that word address.

- **Program Counter:** A 32-bit register that stores the current instruction address.
- **Address alignment:** PC bits [23:2] are used for word-addressed instruction memory, equivalent to PC divided by 4.
- **Instruction memory:** A 16M x 32-bit RAM block configured as read-only for program execution.

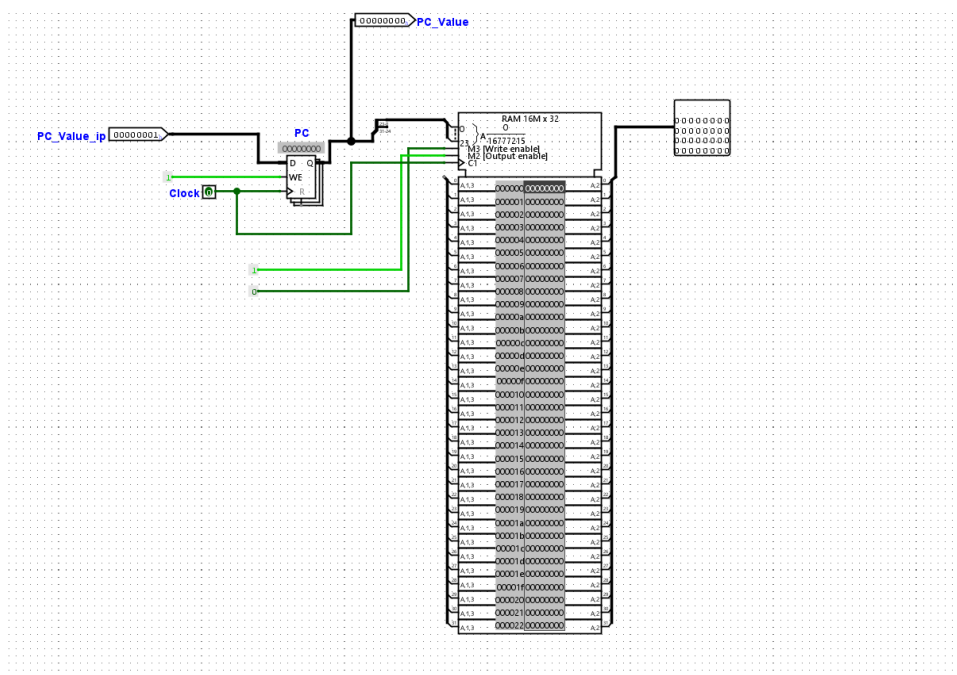


Figure 3: Instruction Fetch unit showing PC register and instruction memory connection.

Why PC[23:2] is used

RISC-V instructions are 4 bytes wide. Removing the lowest two PC bits converts the byte address into a word address, matching the RAM address format used in the circuit.

5. IMMEDIATE GENERATOR MODULE

The Immediate Generator reconstructs immediate values from the instruction bitfields and expands them into a 32-bit signed value. Because RISC-V stores immediate bits differently for different instruction formats, this module separates the I-type, S-type, and B-type paths before selecting the required output.

Instruction Type	Immediate Construction	Used By
I-type	inst[31:20], sign-extended to 32 bits	LW
S-type	{inst[31:25], inst[11:7]}, sign-extended to 32 bits	SW
B-type	{inst[31], inst[7], inst[30:25], inst[11:8], 0}, sign-extended to 32 bits	BEQ

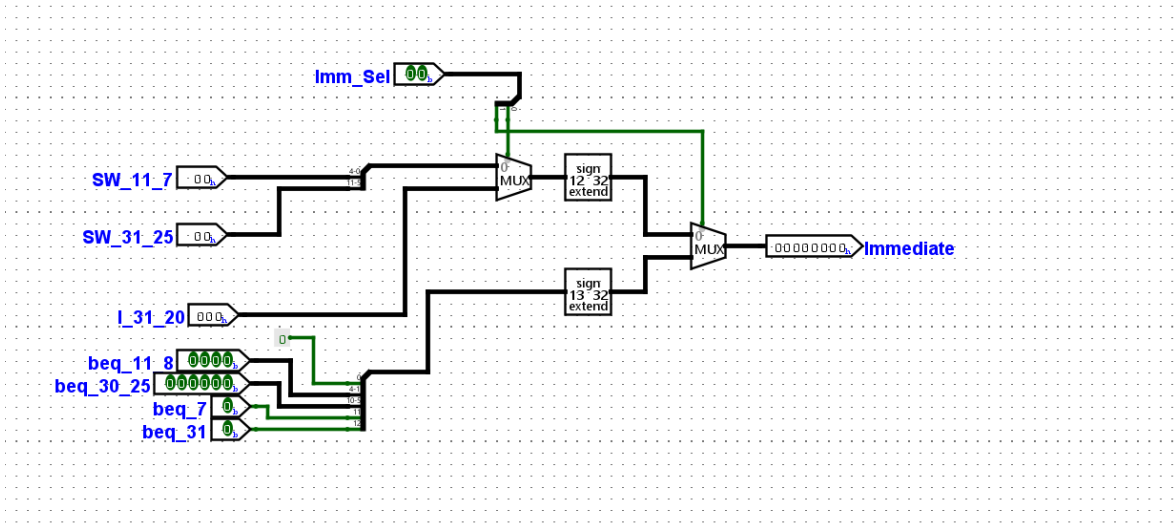


Figure 4: Immediate Generator sub-circuit for I-type, S-type, and B-type immediates.

Branch immediate note

The B-type immediate includes a hardwired zero as the least significant bit. This creates the correct branch offset alignment before the value is added to the PC.

6. DATA MEMORY MODULE

The Data Memory module provides storage for load and store instructions. The ALU calculates the memory address, Read_Data2 provides the value to be stored during SW, and the memory output is selected during LW through the write-back multiplexer.

- **Address input:** Driven by the ALU result after word-address alignment.
- **Write data input:** Comes from the register file second read port, Read_Data2.
- **Control:** MemWrite enables memory writing, while MemRead enables memory reading.

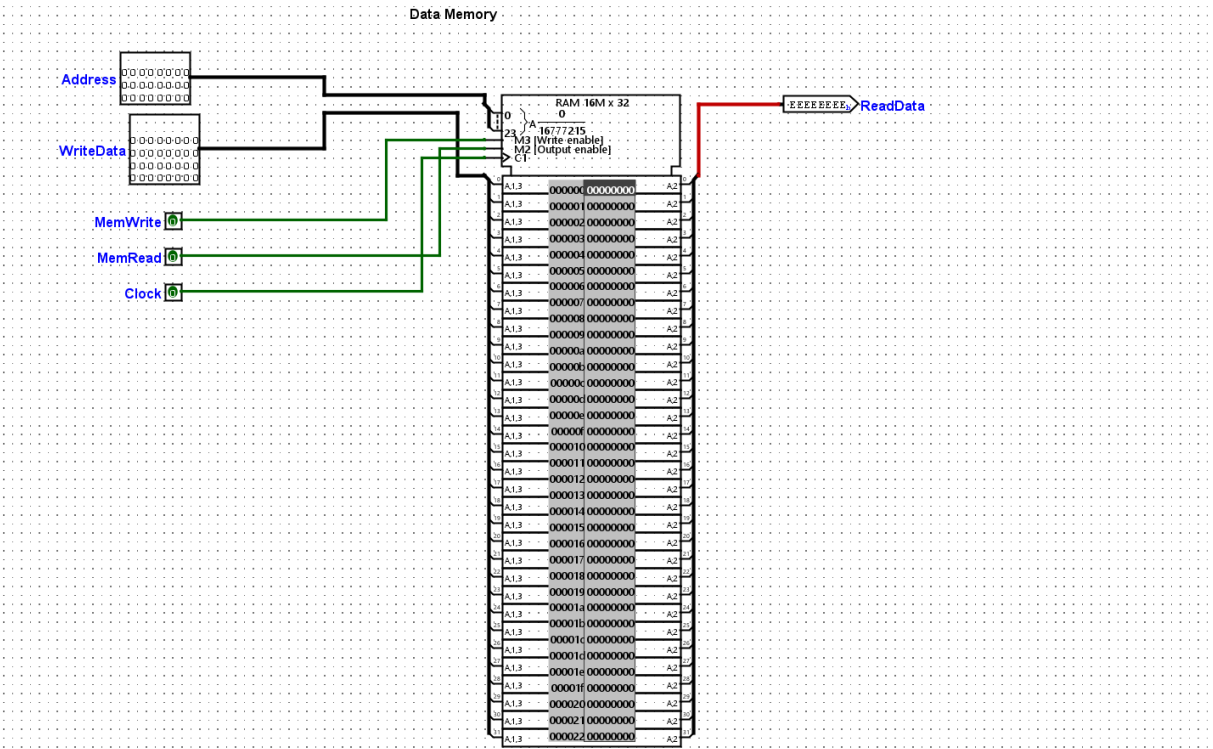
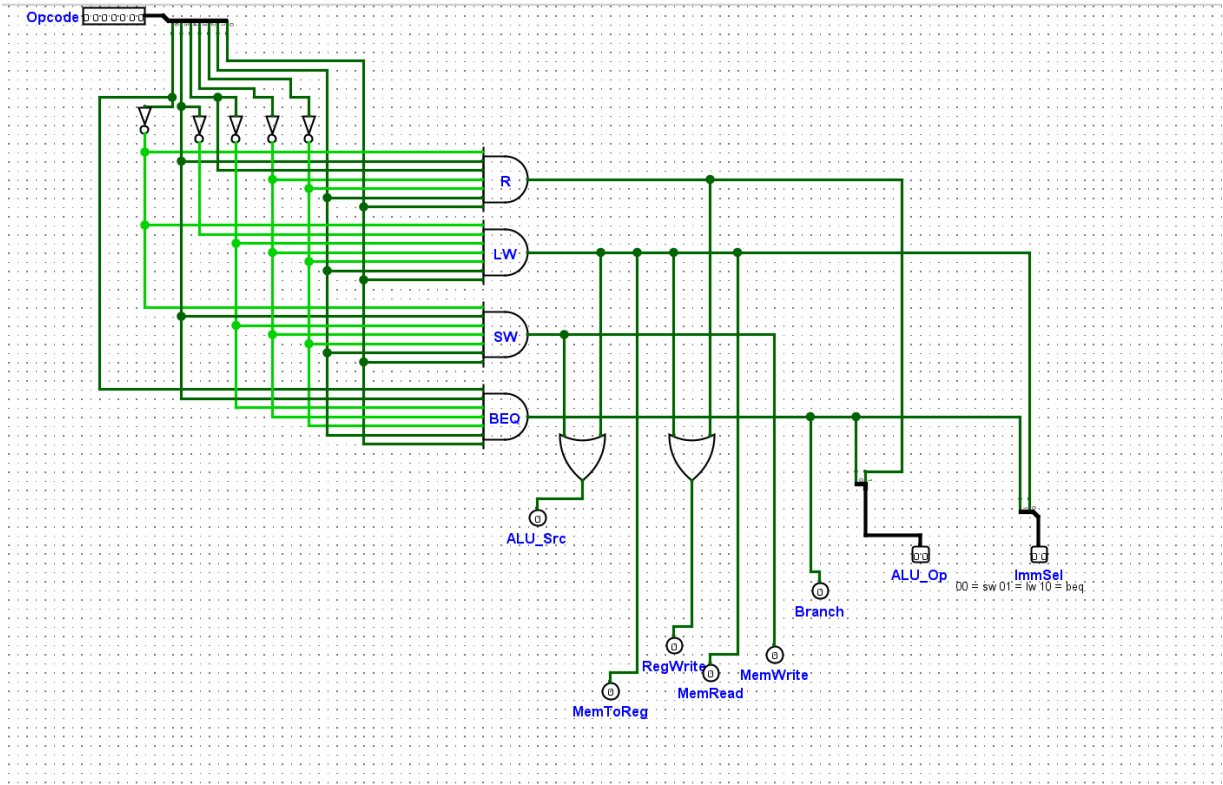


Figure 5: Data Memory sub-circuit used for LW and SW instruction execution.

Item	Specification
Memory size	16M x 32-bit RAM
Load operation	Memory output is written to rd through MemToReg
Store operation	Read_Data2 is written into memory when MemWrite is active
Clock role	Write operation is clock-controlled to preserve stable memory behaviour

7. MAIN AND ALU CONTROL UNITS

The Main Control Unit decodes the 7-bit opcode and generates the high-level control signals that steer the datapath. It identifies R-type, LW, SW, and BEQ instructions and activates the correct combination of RegWrite,



ALUSrc, MemRead, MemWrite, MemToReg, Branch, ImmSel, and ALUOp.

Figure 6: Main Control Unit generating global datapath control signals.

The ALU Control Unit performs the second level of decoding. It uses ALUOp along with funct3 and funct7 information to generate the final 2-bit ALU_operation signal used by the ALU multiplexer.

ALU Operation Code	Target Functional Task
00	Bitwise AND
01	Arithmetic ADD
10	Bitwise OR
11	Arithmetic SUBTRACT

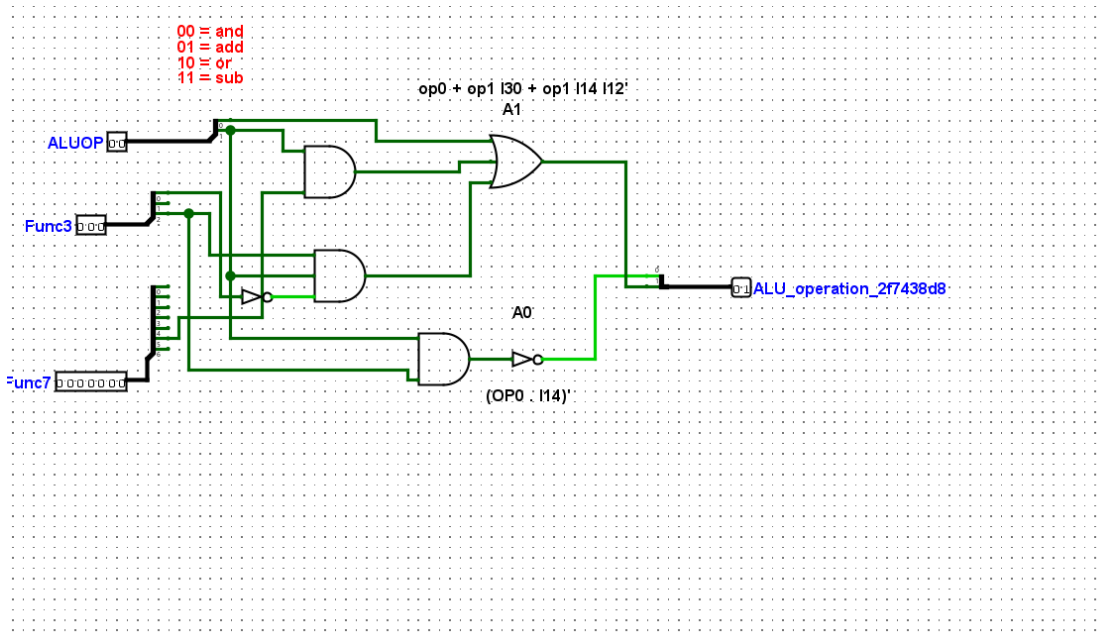


Figure 7: ALU Control Unit translating ALUOp and function bits into ALU operation code.

8. ARITHMETIC LOGIC UNIT (ALU)

The ALU is the execution core of the processor. It receives SRC1 from the register file and SRC2 from either the second register operand or the immediate generator. Multiple operation blocks run in parallel, and the final output is selected by a 4-to-1 multiplexer controlled by ALU_operation.

- **Operational blocks:** 32-bit AND, 32-bit OR, 32-bit adder, and 32-bit subtractor.
- **Zero flag:** A NOR-based network checks whether the ALU result is zero, which is required for BEQ decision logic.
- **Overflow flag:** The sign paths are monitored to indicate overflow during signed arithmetic operations.

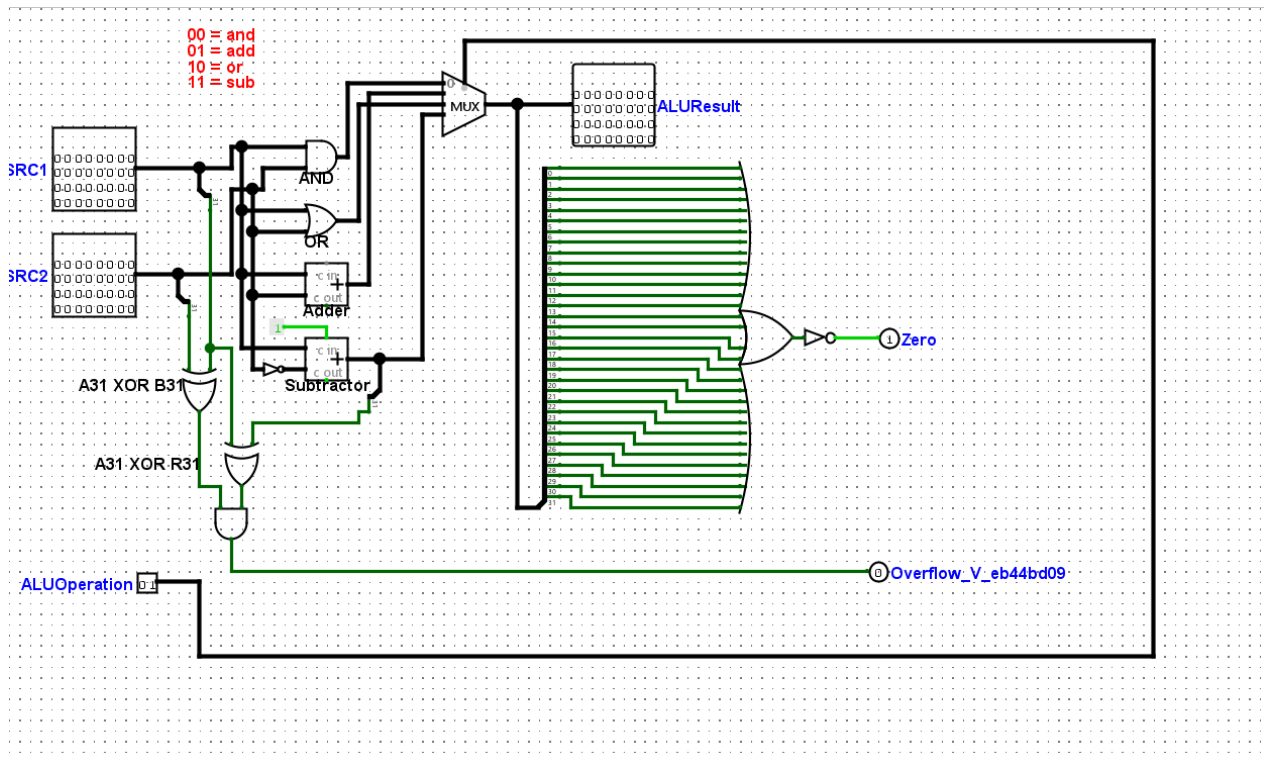


Figure 8: ALU sub-circuit showing arithmetic/logical blocks, output selection, zero flag, and overflow flag.

Branch decision logic

For BEQ, the ALU subtracts the two register operands. If the result is zero, the Zero flag becomes 1. The branch control path then selects the branch target address instead of PC + 4.

9. VERIFICATION SUMMARY

Each module was first checked independently and then integrated into the complete datapath. The final processor design can fetch instructions, decode control signals, read registers, generate immediate values, perform ALU operations, access data memory, update the register file, and select branch targets correctly.

Feature Verified	Instruction Example	Expected Processor Response
Register write-back	add rd, rs1, rs2	ALU result is written to rd when RegWrite is active.
Load word	lw rd, offset(rs1)	Memory output is selected by MemToReg and written to rd.
Store word	sw rs2, offset(rs1)	Read_Data2 is written to data memory at the ALU address.
Branch equal	beq rs1, rs2, label	PC selects branch target when Branch and Zero are both active.

CONCLUSION

The completed Logisim implementation demonstrates a functional single-cycle RISC-V processor with clean modular organization. The design successfully connects the instruction fetch path, register file, immediate generator, ALU, data memory, write-back path, and branch-control logic into one coherent processor datapath.